Birla Institute of Technology & Science, Pilani
Second Semester 2016-2017, CS F111 Computer Programming
Comprehensive Exam (Open Book) *PART A Set X*

Suggested time: 60 minutes **08/5/17, 3.00 - 5.00 PM** **Max. Marks: 44**

NOTE: 1. There are 22 questions in Part A. Each question has exactly one answer correct and carry 2M.
2. There is no negative marking. Overwritten answers will not be rechecked.
3. For each question, write the correct option (A, B, C, or D) at the designated place on the back of this sheet.
4. In each question, choose the best option. Assume required header files are included in each given program.

ID		NAME	

Q1. How many times `hello` will be printed?

```
int main(void) {
 int i,j,k;
 for(i=1; i<=4;i++)
  for(j=1;j<=i;j++)
   for(k=1;k<=j;k++)
  printf("hello\n");
  return 0;
}
```

A) 4 B) 10
C) 20 D) 30

Q2. What is the output?

```
int main(void) {
 char arr[][10] = {"Amit",
"Suresh" , "John" , "Smith"};
 char *ptr[4]; int i;
  for(i=0; i<=3; i++)
    ptr[i] = arr[3-i];
printf("%s,%s",ptr[0], arr[0]);
return 0;
}
```

A) Smith,Amit B) Amit,Amit
C) Smith,Smith D)Runtime error

Q3. What is the output? Assume arr[] begins at address 65686 and size of integer is of 4 bytes.

```
int main(void) {
 int arr[] = {12, 14, 15, 23, 45};
 printf("%u,%u",arr+1,&arr+1);
 return 0;
}
```

A) 65690,65702
B) 65687,65690
C) 65690,65710
D) 65690,65706

Q4. What is the output ?

```
void main() {
 short int a=5;  float b=10;  short int *p;
 p=&b;
 printf("%u\t",p);
 p++;
 printf("%u",p);
}
```

A) Compilation error
B) The difference in printed addresses is sizeof(short int)
C) The difference in printed addresses is sizeof(float)
D) Segmentation fault run time error.

Q5.What is the output?

```
void fun(int x) {
 if(x>1) {
   fun(--x);
   printf("%d",x);
   fun(--x);
 } }
void main() {
 fun (4);
}
```

A) 1231 B) 1321
C) 1234 D) 4321

Q6. What is the output?

```
int main (void)
{
 char c[] = "KATEWINCE";
 char *p =c;
 printf("%s", p+p[3]-p[1]) ;
 return (0);
}
```

A) INCE B) EWINCE
C) WINCE
D) None of the above

Q7. What is the output?

```
void main(void) {
 int i  = 0;
for(printf("A");printf("B");printf("C"))
{
 for(printf("D");printf("E");printf("F"))
   break;
 if (i--)  break;
 }
}
```

A) ABDECBFE B) ABDECBDE
C) ADCBD D) Infinite Loop

Q8 What is the output?

```
int main() {
 union temp { int a, b;};
 union temp t;
 t.a=100;
 t.b=200;
 printf("%d",t.a);
 return 0;
}
```

A) Compilation error B) 100
C) Run time error D) 200

Q9. Consider the given program:

```
typedef struct {
 int age;
 float marks;
} stu;
int main() {
  stu arr[5];
  stu *ptr1 = arr;
  stu *ptr2 = ptr1 + 5;
  LINE1
  printf ("%d",temp);
}
```

If the given program is required to print the size of array arr[], which of the following two statements can be substituted in place of LINE1.

S1: int temp = (char*) ptr2 - (char*) ptr1;
S2: int temp = (ptr2 - ptr1) * sizeof(stu);

A) Only S1
B) Only S2
C) Any one of S1 or S2
D) Neither S1, nor S2

Q10. Which of the following doesn't require traversal of elements in a linked list, given a pointer to the first node of the list?
A) Deleting an element at the end of a singly linked list
B) Deleting an element from the end of a linear doubly linked list.
C) Inserting an element after the last element in a circular doubly linked list.
D) Inserting an element at the end of a linear singly linked list.

Q11. In C language, if precedence of + operator is higher than * operator; and operator + is right associative, the value of following expression is:

E: **5 * 4 + 7 * 8 + 9**

A) 225 B) 449
C) 85 D) 935

Q12. The decimal equivalent for the following IEEE floating point number is:
1 10000000 10110000000000000000000
A) -1.175 B) -3.375 C) -1.6875
D) None of the above

Q14. Assuming 2's complement representation, if $X = (A3FD)_{16}$ and $Y = (9CE)_{16}$, then (X+Y) in base 16 is:
A) F8DCB B) 8DCB C) DCB
D) None of the above

Q13. What is the output?

```
int main (void) {
 enum tak {_2 , _3 , _4, _1};
 enum cak {_5,_6,_7 };
 float b;
 int a;
 a = _3*6/_7%(_1-1);
 b = 2*(_3+_5);
 printf ("a=%d b=%d\n",a,b);
}
```

A) a = 0 b = 1
B) a = 1 b = 0
C) Compile time error
D) None of the above

Q15. What is the output ?

```
void main()   {
char *str = "BITS Pilani";
char *str1 = "BITS PILANI";
if (strcmp(str, str1)<0)
  printf("LOWER");
else
  printf("UPPER");
 }
```

A) Nothing is printed
B) UPPER
C) LOWER
D) Compilation error

Q16. What is the output?

```
char writeBackward(char s[], int n) {
  if (n == 0)
    return '\0';
  writeBackward(s, n--);
  printf ("%c",s[n]);
}
void main() {
   writeBackward("BITS PILANI",11);
}
```

A) BITS PILANI B) Compile time error
C) INALIP STIB D) Run time error

Q17. What is the output?

```
int main() {
 int i,j,k;
 for(i=1;i<=3;i++)   {
  if(i%2==0)
    k=2;
  else
    k=1;
  for(j=1;j<=i;j++,k+=2)
    printf("%d", k);
 }
 return 0;
}
```

A) 213
B) 124135
C) 124
D) None of the above.

Q18. Convert the following from base 6 to base 8:

$(524)_6 = (?)_8$

A) 304
B) 104
C) 404
D) None of the above

Q19. What is the output?

```
int a, b, c = 0;
void foo (void);
void main () {
 static int a = 1;
 foo();
 a += 1;
 foo();
 printf ( "%d %d ", a,b) ;
}
```

```
void foo (void)
{
   static int a = 2;
   int b = 1;
   a += ++b;
  printf ("%d %d ", a,b);
}
```

(A) 3 1 4 1 4 2 (B) 4 2 6 1 6 1
(C) 4 2 6 2 2 0 (D) 3 1 5 2 5 2

Q20. Select the correct choice.

```
void main() {
   typedef struct
   {
      int age;
      char *name;
   } stu;
   stu amit;
   amit.age = 18; //LIN1
   strcpy (amit.name, "amit"); //LIN2
printf ("%s %d",amit.name,amit.age);
}
```

A) Program prints **amit 18**
B) If **amit 18** if to be printed, replace LIN2 with: **amit.name = "amit";**
C) If **amit 18** is to be printed, insert the following after LIN1 and before LIN2:

```
amit.name=
          (char*)malloc(5*sizeof(char));
```

D) Both (B) and (C) above.

Q21. The following program intends to find the median (in variable t) of n sorted elements stored in array arr. What should be replaced in place of C and S to achieve the task?

```
  void main() {
    int n, i;
    float t;
    scanf("%d",&n);
    int arr[n];
    //assume arr[] is initialized here.
    if (C)
      S
    else
      t = arr[n/2];
      printf ("Median = %f",t);
  }
```

A) C: n%2==0
   S: t = (arr[n/2] + arr[n/2+1])/2.0;
B) C: (2*n)%2 == 0
   S: t = (arr[n/2] + arr[n/2+1])/2.0;
C) C: n%2==0
   S: t = (arr[n/2] + arr[n/2-1])/2.0;
D) C: (2*n)%2 == 0
   S: t = (arr[n/2] + arr[n/2-1])/2.0;

Q22. Which of the geometric patterns can be ***best associated*** with the output of the following program.

```
void main() {
   int i, j, rows;
   scanf("%d", &rows);
   for(i=1; i<=rows; i++) {
     for(j=1; j<=rows; j++)  {
        if(i==1 || i==rows || j==1 || j==rows)
           printf("*");
        else
           printf(" ");
     } //inner for loop
     printf("\n");
   }//outer for loop
}
```

A) two vertical parallel lines
B) two horizontal parallel lines
C) hollow square
D) hollow triangle

PART A Set X
ANSWER AREA (OPEN BOOK)
Write your answers legibly in the space provided below.

Q. No.	1	2	3	4	5	6	7	8	9	10	11
Correct Option											
Q. No.	12	13	14	15	16	17	18	19	20	21	22
Correct Option											

Total Correct Answers: [ ] x 2 = *Total Marks*

RECHECK:

Birla Institute of Technology & Science, Pilani
Second Semester 2016-2017, CS F111 Computer Programming
Comprehensive Exam (Open Book) *PART B*

Date: May 08, 2016 **Time: 3.00 - 5.00 PM** **Max. Marks: 46**

NOTE: Each question is partially implemented. ***Write the missing parts as your answers at the designated place in the separate answer sheet provided. Assume suitable header files are included. In the ans sheet, each box will contain only one statement, unless otherwise mentioned.***

Q1. A number N is a perfect square if there exists a number k, such that K^2=N. e.g. 16 = 4^2, hence 16 is a perfect square. Also, an n digit number M is an Armstrong number if sum of each digit raised to the power the number of digits (i.e. n) is equal to M itself. e.g. 1634 = $1^4+6^4+3^4+4^4$, hence 1634 is an Armstrong number. The program shown in Figure-1 takes a number as an input from the user and checks whether the number is perfect square. It also checks whether this input number is an Armstrong number or not. Write the missing code to complete the program.

```
int main() {
 int no , i;
 scanf("%d",&no);
 for(i=0 ; i<=no ; i++) {
  if (     Q1(a)      )
   printf("perfact square\n");
} //end for
if( ___Q1(b)___ )
  printf("Not perfact square\n");
int digit =0, temp = no;
while(temp!=0) {
     Q1(c)       ;
 temp = temp/10;
} //end while
temp = no;
int rem=0,total=0;
while (temp!=0) {
     Q1(d)       ;
 total += pow ( rem , digit );
     Q1(e)       ;
} //end while
if( ___Q1(f)___ )
  printf("armstrong number");
else
 printf("not armstrong number");
return 0;
}
```

Figure-1

Q2. A file named "file.txt" contains multiple sentences, where each sentence contains multiple English words separated with one white space. A program is written to construct a linked list, named **list** of all the distinct words of file (refer to structure definition of **WORDS**, or **node2**). Also, for each distinct word, the corresponding node of **list** also maintains another linked list with its nodes containing line number (**lno**) on which the word has occurred in the file and its frequency (**freq**) in that line, i.e., how many times a word occurred in that line (refer to structure definition of **LINE**, or **node1**). For example, if the contents of "file.txt" are as shown in Figure-2, corresponding linked list is shown in Figure-3. Complete the following two functions (*partial implementation given in Figure-4 and Figure-5*) to achieve the above task:

(a) WORDS* insert_word (WORDS *list, char *temp, int ln_no)

This function takes as input a pointer to the linked list of words (**list**), the word to be inserted (**temp**), and line number (**ln_no**) on which the word has appeared in file. The function inserts every new distinct word at the end of the **list**. If the word to be inserted (i.e. **temp**) already exists in the list, then there are two cases: Case-1: The word appeared on the same line (**ln_no**) on which it has previously appeared; Case-2: The word appeared in a new line.

(b) WORDS* delete_words (WORDS *list, int f_low, int f_high)

This function deletes all the words from the linked list whose total number of occurrences in the word list (**list**) is in between **f_low** and **f_high** (both inclusive) and returns modified word list. If no such word lies in the word list with desired occurrence frequency, then function returns unmodified word list. [**NOTE: deletion of a node means updating the pointers and deallocating the memory**].

Figure-2: words stored in "file.txt"

```
aa cc eee aa
cc aa bb aa
cc cc
```

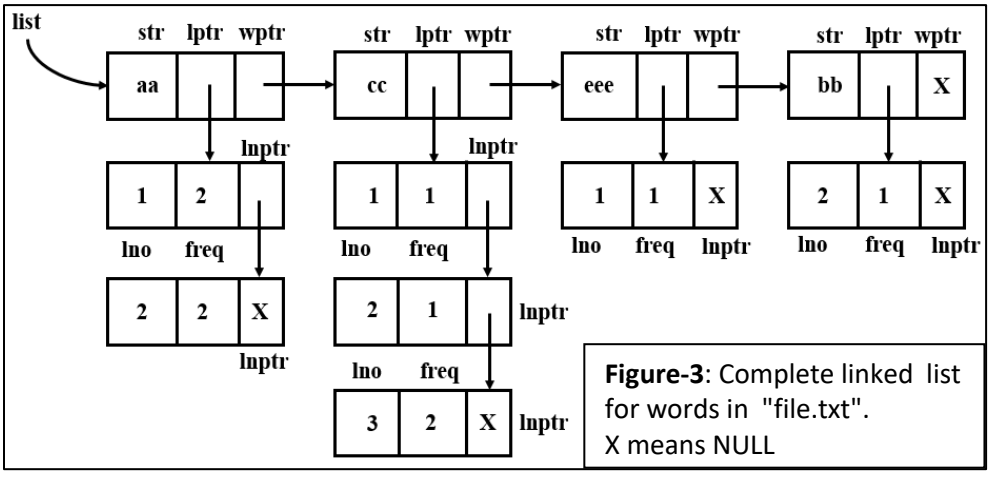

Figure-3: Complete linked list for words in "file.txt". X means NULL

```
/* TYPE DEFINITIONS for Q2*/
typedef struct node1 LINE;
typedef struct node2 WORDS;
struct node1
{
  int lno;
  int freq;
  struct node1 *lnptr;
};
struct node2
{
  char str[50];
  struct node1 *lptr;
  struct node2 *wptr;
};
```

```
WORDS* insert_word(WORDS *list, char *temp, int ln_no)
{
  WORDS *wt,*pwt;
  wt = pwt = list;
  while(wt != NULL)
  {
     if(strcmp(wt->str,temp)==0)
     {
       LINE *lt = wt->lptr;
       LINE  *plt = wt->lptr;
       while(lt != NULL)
       {
         if(   Q2(a)   )
         {
            lt->freq = lt->freq+1;
            return list;
         }
         plt = lt;
         lt = lt->lnptr;
       } //inner while close
       if(lt==NULL)
       {
         LINE *newline;
        Q2(b) //More than one statement may come here
       }
     }//if close
     else
     {
           pwt = wt;
           wt = wt->wptr;
     }
   }//outer while close
  WORDS *newword;
  LINE *newline;
  newword = (WORDS*)malloc(sizeof(WORDS));
  newline = (LINE*)malloc(sizeof(LINE));
  strcpy(newword->str,temp);
  newword->wptr = NULL;
    Q2 (c) //More than one statement may come here

  if(list==NULL)
     Q2(d)      ;
  else
     Q2(e)      ;
 return list;
}
```

Figure-4

```
WORDS* delete_words(WORDS *list, int f_low, int f_high)
{
/* deletion of a node means updating the pointers and
deallocating the memory*/
  int f_count;
  WORDS *w,*pw;
  LINE *l, *pl;
  w = pw = list;
  while(w != NULL) // outer while loop
  {
    f_count=0;
    l = w->lptr;
    while(l!=NULL)
    {
          f_count =    Q2(f)    ;
          l = l->lnptr;
    }
    if(f_count >=f_low && f_count <= f_high)
    {
          pl = l = w->lptr;
      while(l!=NULL)
      {
        Q2(g) //More than one statement may come here
      }
       if(w==list)
       {
         Q2(h) //More than one statement may come here
       }
       else
       {
         Q2(i) //More than one statement may come here
       }
     }
     else
     {
          pw = w;
          w=w->wptr;
     }
  }//end of outer while loop
  return list;
}
```

Figure-5

Q3. A file named *"input.txt"* contains the information of the state of the game of scrabble implemented using 2D array of characters (only upper case alphabets). Each row in the file contains a ***word, row index, column index and a binary digit*** separated using a white space character. For every row, the word is to be placed on the scrabble (i.e. 2D array) starting at position ***(row index, column index)*** and the ***binary digit*** indicates the direction in which the word is to be placed (a value of **1** indicates **horizontal direction towards the east**, and a value of **0** indicates **vertical direction towards the south**).
Suppose the file input.txt contains four records as shown in **Figure 6**, then the scrabble should look like the 2D array in **Figure 7**.

```
COMPUTER 3 2 1
PROGRAM 1 3 0
EASY 6 2 1
RIGHT 3 9 0
```

Figure 6

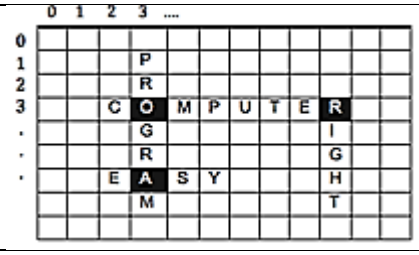

Figure 7

The following program (Figure-8) is meant to accomplish the above mentioned task as well as to print the common intersecting characters of every two intersecting words on the scrabble. For the above example, the program should print the characters O , R and A on separate lines. Complete the missing pieces of lines.

```
#define ROW 20
#define COL 20
#define MAXENTRY 20
typedef enum key { DOWN, RIGHT} Dir;
typedef struct game
{
   char *word;
   int row;
   int col;
   Dir dir;
}Entry;
int main()
{
   char scrabble[ROW][COL];
   int visit[ROW][COL]={0}; //initialize all elements to 0
   int i=0,j,k,p, unique=0, cnt;

   FILE *fp = ___Q3(a)___;
   Entry node[MAXENTRY];

   // Read from file
   while(!feof(fp))
   {
     node[i].word = (char*) malloc(sizeof(char)*(COL+1));
     ___Q3(b)___ //Read the word from file
          i++;
   }
   fclose(fp);
   cnt = i;
   for(i=0; i<ROW;i++)
      for(j=0; j< COL;j++)
         scrabble[i][j] = '-';
```

```
// Populate scrabble now
for(i=0;i<cnt;i++)
  {
     int r = node[i].row;
     int c = node[i].col;
     if( ___Q3(c)___ ) {
        for(j=0;j<strlen(node[i].word); j++)
           ___Q3(d)___;
     }
     else {
        for(j=0;j<strlen(node[i].word); j++)
           ___Q3(e)___;
     }
  }
// Print the characters at the intersection of two words
for(i=0;i<cnt;i++)
  for(j=0;j<cnt;j++)
  {
    int r1 = node[i].row, c1 = node[i].col;
    int r2 = node[j].row, c2 = node[j].col;
    if( ___Q3(f)___ )
       for(k=0;k<strlen(node[i].word);k++)
          for(p=0;p<strlen(node[j].word); p++)
             if( ___Q3(g)___ )
                visit[r1][c1+k]=1;
    if( ___Q3(h)___ )
       for(k=0;k<strlen(node[i].word);k++)
          for(p=0;p<strlen(node[j].word); p++)
             if( ___Q3(i)___ )
                visit[r1+k][c1]=1;
  }
  for(i=0;i<ROW;i++)
     for(j=0;j<COL;j++)
        if(visit[i][j])
           printf("\n%c", scrabble[i][j]);
return (0);
}
```

Figure-8